

Portfolio

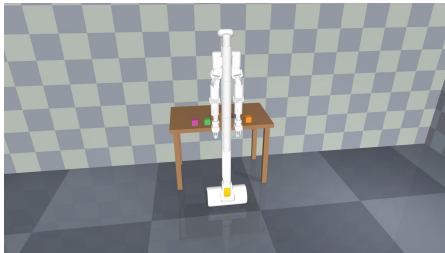
Derek Wang | derek.wang1@uwaterloo.ca | github.com/4appleSA0CE | derekwng.ca

MR. CLEAN: LLM-ORCHESTRATED MOBILE MANIPULATION

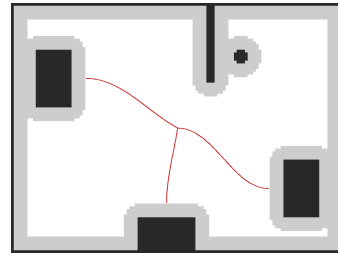
[GitHub](#) | [Demo](#) | [Devpost](#)

Python, MuJoCo, PyTorch, Stable-Baselines3, ROS 2, Claude API

Self-balancing robot with two wheels and two 7-joint arms. Tidies three tables in a simulated MuJoCo room from one typed prompt, with a different controller at each table.

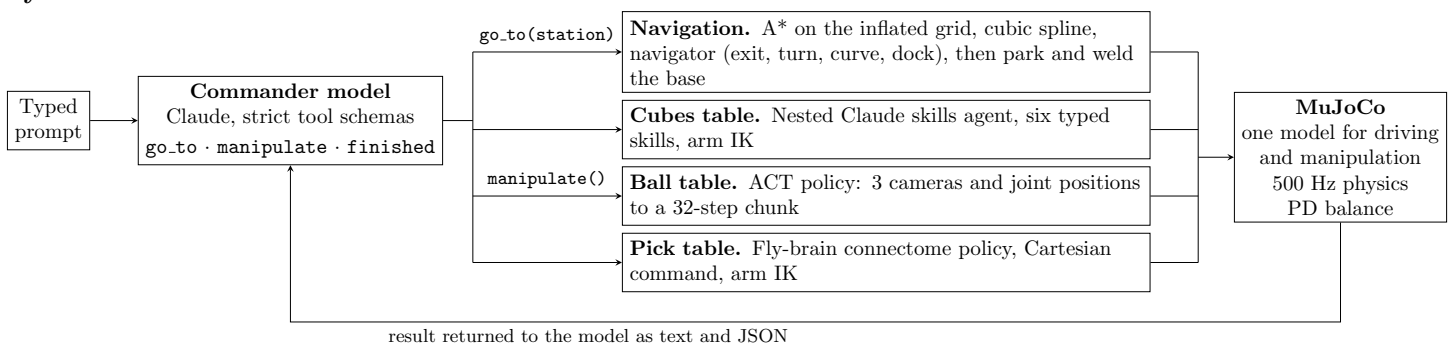


The robot docked at the cube table in MuJoCo



Occupancy grid, footprint inflation (grey) and planned routes (red)

System architecture



- `manipulate()` runs the controller that belongs to the table where the robot is parked; on arrival the base is welded in place and the contact stiffness raised, which gives the arms a fixed platform
- No fallback controllers, so a failed station is reported as failed

Claude agent controller

- Two levels, both with strict JSON tool schemas; the nested agent at the cube table has six skills: `look`, `pick(object, arm)`, `place(into)`, `home`, `give_up(object, why)`, `finished`
- Model never receives coordinates; each skill returns a short note, a failure cause from a fixed list, and a symbolic description of the scene
- Code selects the arm and retries a grasp up to 8 times inside the skill, so the model decides only the order of the actions
- Unknown object names rejected inside the skill with `object_not_found` and the list of valid objects
- Limits per run: 2 repeats of an identical failed call, 40 nested steps, 12 top-level steps, no second attempt on an object after `give_up`
- Live run with `claude-fable-5-1`: **4 of 4 cubes sorted in 13 tool calls**

Navigation

- Classical planner for the base, because the robot must not fall and the learned pilot has no obstacle avoidance
- Occupancy grid with 5 cm cells, obstacles inflated by 0.26 m for the robot footprint
- 8-connected A* without corner cutting, line-of-sight waypoint removal, then a **clamped cubic spline** through the remaining waypoints
- Four route phases (exit, turn, curve, dock); replans at each phase boundary and when the base is more than 15 cm off the path
- All **9 test routes** arrive within 2.4 to 9.6 cm and 1.8 to 4.9 degrees of the goal, with no falls and no furniture contact

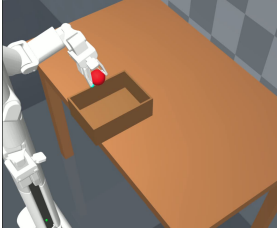
ACT controller

- **Action Chunking Transformer** (Zhao et al., 2023), small enough to train on a laptop
- Input: three 128 px camera images (head and two wrists) and 16 joint positions. Output: a chunk of 32 joint commands at 20 Hz, which is 1.6 s of motion
- One ImageNet ResNet-18 shared across the cameras, into a transformer with 4 encoder and 4 decoder layers (width 256, 8 heads); 21.9 M parameters

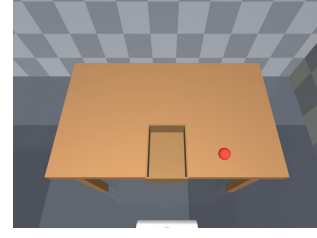
- CVAE encoder supplies a 32-dimensional latent z in training, with $z = 0$ at inference. Loss:

$$\mathcal{L} = \|\hat{a}_{1:32} - a_{1:32}\|_1 + 10 D_{\text{KL}}(q(z | s, a_{1:32}) \| \mathcal{N}(0, I))$$

- Overlapping chunks blended with weights $w = \exp(-0.01(t - t_0))$; trained on 80 scripted-teacher demonstrations with random ball and crate positions



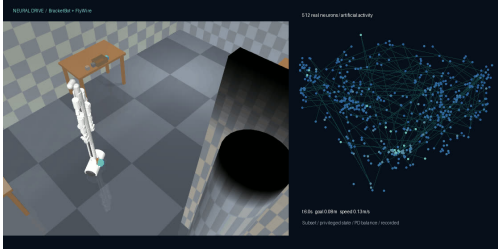
ACT carrying the ball to the crate



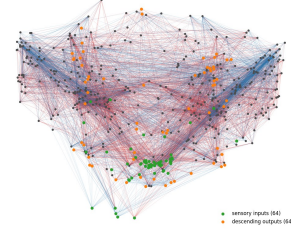
Head camera view, one of three policy inputs

Why ACT has the lowest success rate

- On new layouts the policy closes on the ball about 4 times in 10 and gets it into the crate **2 times in 14**; most failures are a ball dropped during the carry or a grasp about 0.5 s too early
- **Base pose.** Demonstrations recorded with a fixed base. In the full demo the robot parks 2 to 10 cm and up to 5 degrees from that pose, so each camera sees a shifted scene
- **Model.** Demonstrations used 0.005 armature on the gripper joints. The demo model does not have it, because it causes the cube skills to fail, so it is set only for the ACT episode. Contact pads and solver settings also change when the base parks
- **Rendering.** A recalculated model constant moved the near plane of each camera from 80 mm to 230 mm and removed the gripper and the ball from the wrist images; 0 of 10 episodes until corrected
- **Data.** Small data set. The scripted teacher succeeds about 7 times in 10 and drops the ball in one third of its carries, because flat gripper pads do not hold a sphere well



Point-goal pilot driving the base, with live neuron activity



The 512-neuron subgraph at real neuron positions (red excitatory, blue inhibitory)

Fly-brain connectome controller

- Fruit-fly brain wiring used as the feature extractor of a reinforcement-learning policy
- Source: **FlyWire FAFB v783 connectome** (139,255 neurons, 2.7 M connections). Kept the 64 sensory, 64 descending and 384 central neurons with the most synapses: **512 neurons and 8,688 edges**, wiring frozen
- Synapse counts c_{ij} become signed weights, negative if the presynaptic neuron releases GABA or glutamate, with each row normalised:

$$\tilde{A}_{ij} = s_j c_{ij}, \quad A_{ij} = \tilde{A}_{ij} / \max\left(1, \sum_k |\tilde{A}_{ik}|\right)$$

- Observation enters the sensory neurons as an input current I and is propagated for four rounds, with four channels per neuron:

$$H^{(t+1)} = \tanh(0.35 H^{(t)} + G \odot (A H^{(t)}) M^\top + I + B)$$

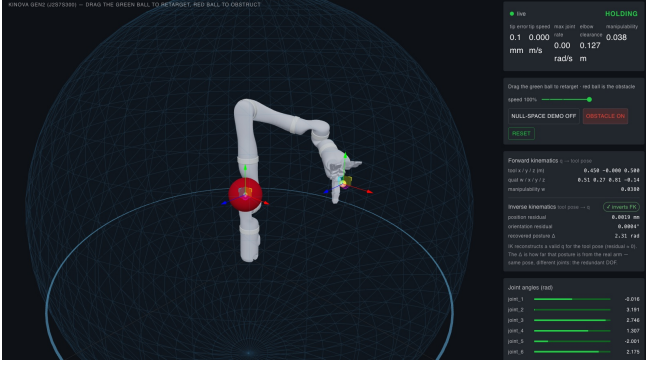
- Descending-neuron states give a 256-dimensional feature vector for a Stable-Baselines3 policy head
- Trained parts: observation encoder, per-neuron gain G and bias B , 4×4 channel mix M , and the head. Behaviour cloning from a scripted teacher, then optional PPO

Held-out point goals (20 seeds, 0 falls)	Cloned	Cloned + PPO
Fly connectome features	20/20	17/20
MLP features	18/20	20/20

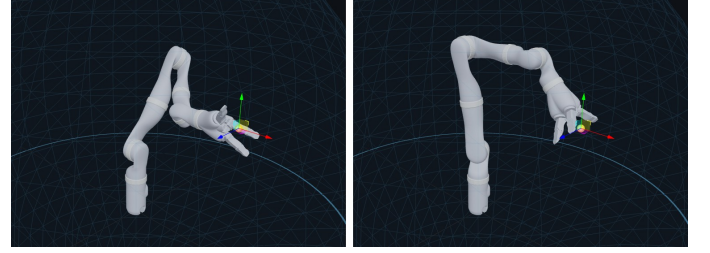
- Full connectome needs 1.2 s per forward pass on a CPU against a 50 ms control period, so the graph was reduced to 512 neurons
- An MLP gets almost the same results, so no advantage from the fly wiring is shown yet; a shuffled-graph ablation is not yet done

C++17, Eigen, MuJoCo, CMake, React, TypeScript | **No KDL, Pinocchio or MoveIt**

Motion-control stack for a 7-joint Kinova Gen2 arm in C++17 with Eigen. All kinematics and dynamics code is in the repository, with no robotics library. MuJoCo supplies only the physics simulation and the mesh collision check.



Browser dashboard: drag the target or the obstacle and the arm replans



Null-space motion: two postures, tool held to 0.2 mm

- **Redundancy.** Controller tracks the tool position, which uses 3 of the 7 degrees of freedom; the other 4 keep the elbow away from obstacles and joint limits while the tool stays on the target
- **Rotations and rigid motions.** $SO(3)$ and $SE(3)$ with closed-form maps and separate branches for small angles and for angles near π :

$$e^{\hat{\omega}} = I + \frac{\sin \theta}{\theta} \hat{\omega} + \frac{1 - \cos \theta}{\theta^2} \hat{\omega}^2, \quad \text{Ad}_T = \begin{bmatrix} R & 0 \\ \hat{p}R & R \end{bmatrix}$$

- **Forward kinematics.** **Product of exponentials** in body form with home pose M and body screws B_i , so the Jacobian and the error twist $\log(T^{-1}T^*)$ are both in the tool frame:

$$T(q) = M \prod_{i=1}^7 e^{[B_i] q_i}, \quad J_{b,i} = \text{Ad}_{(e^{[B_{i+1}] q_{i+1}} \dots e^{[B_7] q_7})^{-1}} B_i$$

- **Inverse kinematics.** **Damped least squares** on the position Jacobian J_p with a null-space term, $\lambda = 10^{-3}$. Steps stay bounded near singularities, and 26 fixed seeds make each plan reproducible:

$$J^\# = J_p^\top (J_p J_p^\top + \lambda^2 I)^{-1}, \quad \Delta q = J^\#(p^* - p) + (I - J^\# J_p) \dot{q}_0$$

- **Secondary objectives.** $\dot{q}_0$ sums four terms: face the target, avoid the joint limits, clear the obstacle, clear the scenery. Clearance terms weighted by $w_k = \max(0.3, 1 - k/K)$, because at full weight they have no equilibrium when the requested clearance is not possible and the solver does not stop
- **Dynamics.** **Recursive Newton-Euler** with twists and wrenches: a forward pass for the link twists V_i and accelerations, a backward pass for the wrenches and joint torques:

$$F_i = \text{Ad}_{T_{i+1,i}}^\top F_{i+1} + \mathcal{G}_i \dot{V}_i - \text{ad}_{V_i}^\top \mathcal{G}_i V_i, \quad \tau_i = F_i^\top A_i$$

Gravity $g = \text{RNE}(q, 0, 0)$, Coriolis term $\text{RNE}(q, \dot{q}, 0) - g$, columns of $M(q)$ from unit accelerations

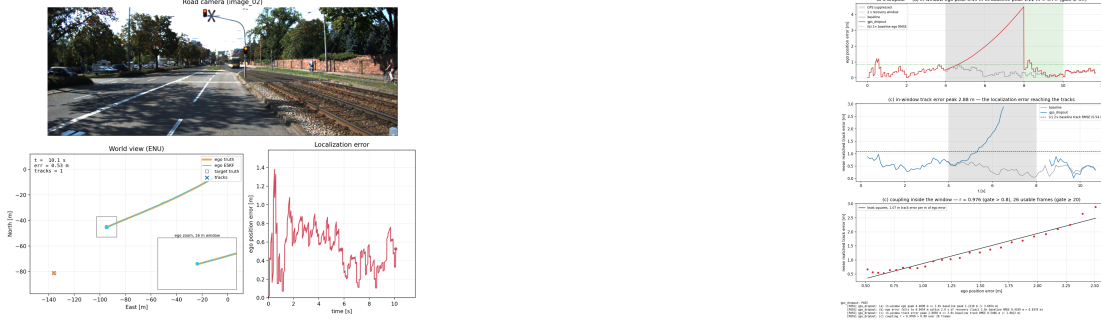
- **Trajectory.** Straight line in joint space, $q(s) = q_0 + s \Delta q$. Velocity, acceleration and jerk limits of each joint mapped onto s before the move starts, for example $\dot{s}_{\max} = \min_i v_i / |\Delta q_i|$, so no joint can exceed a limit
- **Control.** **Computed torque at 1 kHz** with critically damped gains ($K_p = 196$, $K_d = 28$, $K_i = 30$), friction feedforward, and anti-windup by conditional integration:

$$\tau = (M(q) + 0.1I) \left(\ddot{q}_r + K_d \dot{e} + K_p e + K_i \int e dt \right) + C(q, \dot{q}) \dot{q} + g(q) + \tau_{\text{fric}}$$

- **Verification.** Forward kinematics and inverse dynamics compared with MuJoCo, Jacobian with finite differences, on 1,000 random configurations each
- **Results.** 50 motion scenarios with no actuator saturation and no joint reversal. Tip error **0.2 mm or less on 46 of the 49 reachable targets**; largest 6.7 mm on a sweep point near the base, 6.0 mm on a retarget across the base
- **Design choice.** IK solved once when the goal or the obstacle moves, never inside the 1 kHz loop. An earlier version solved it each millisecond, needed a 2 cm tolerance, and the arm vibrated
- **Limits.** Gradient projection moves the arm only along its self-motion manifold, so clearance is not guaranteed in all directions. Simulation only

C++, Eigen, ROS 2, Python, Docker, KITTI | **0.558 m mean error over 6.6 km** | **0.886 MOTA**

Localization filter and multi-object tracker for a car, in C++ on ROS 2, tested on KITTI. The tracker places detections in the world frame with the pose estimate of the filter, so the localization error propagates into the tracking error.



Filter estimate against the KITTI reference, drive 0001

GPS cut for 4 s: track error follows ego error ($r = 0.976$)

- **Error state.** **15-state error-state Kalman filter** in an ENU map frame with a global attitude error:
 $\delta x = [\delta p, \delta v, \delta \theta, \delta b_a, \delta b_g] \in \mathbb{R}^{15}$, $q_{\text{true}} = \text{Exp}(\delta \theta) \otimes q$
- **Mechanization.** Nominal state driven by the 100 Hz IMU, with $a = a_m - b_a$ and $\omega = \omega_m - b_g$:

$$p \leftarrow p + v \Delta t + \frac{1}{2}(Ra + g)\Delta t^2, \quad v \leftarrow v + (Ra + g)\Delta t, \quad q \leftarrow q \otimes \text{Exp}(\omega \Delta t)$$

- **Covariance propagation.** $P \leftarrow F_x P F_x^\top + Q_d$ with

$$F_x = \begin{bmatrix} I & I\Delta t & -\frac{1}{2}[Ra]_\times \Delta t^2 & -\frac{1}{2}R\Delta t^2 & 0 \\ 0 & I & -[Ra]_\times \Delta t & -R\Delta t & 0 \\ 0 & 0 & I & 0 & -R\Delta t \\ 0 & 0 & 0 & I & 0 \\ 0 & 0 & 0 & 0 & I \end{bmatrix}, \quad Q_d = \text{diag}(0, \sigma_a^2 \Delta t^2, \sigma_g^2 \Delta t^2, \sigma_{b_a}^2 \Delta t, \sigma_{b_g}^2 \Delta t)$$

- **GPS update.** 10 Hz, lever arm ℓ , $H = [I \ 0 \ -[R\ell]_\times \ 0 \ 0]$. Gain $K = PH^\top S^{-1}$ from an LDLT solve, Joseph-form covariance update, then error injection and reset
- **Tracker.** **Interacting multiple model** filter for each target over constant velocity, constant acceleration and two constant-turn models at ± 0.2 rad/s. With Markov matrix Π ($\Pi_{ii} = 0.97$), innovation y_j and its covariance S_j :

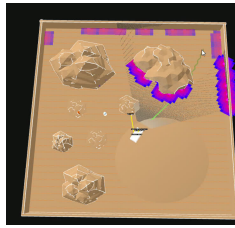
$$c_j = \sum_i \Pi_{ij} \mu_i, \quad \mu_{i|j} = \frac{\Pi_{ij} \mu_i}{c_j}, \quad \mu_j \leftarrow \frac{\mathcal{N}(y_j; 0, S_j) c_j}{\sum_k \mathcal{N}(y_k; 0, S_k) c_k}$$

- **Association.** Cost $1 - \text{IoU}_{3D}$ with a Jonker-Volgenant assignment; track confirmed after 3 hits and deleted after 3 consecutive misses
- **Coupling.** Each detection transformed with the filter pose at the exact detection timestamp, $p_e = \hat{R} p_b + \hat{t}$

Sequence	s	ATE (m)	GPS only	NEES in band	Sequence	s	ATE (m)	GPS only	NEES in band
09_26_0001	11.7	0.567	1.310	94.1%	09_30_0020	113.6	0.503	1.305	94.2%
09_26_0009	27.6	0.449	1.288	94.3%	09_30_0033	166.0	1.013	1.293	56.4%
09_26_0015	31.3	0.436	1.264	95.8%	10_03_0042	121.9	0.575	1.298	87.1%
09_26_0117	36.1	0.472	1.263	94.7%	09_29_0004	35.9	0.448	1.274	95.2%
					Mean		0.558	1.287	

- **Localization.** **0.558 m** mean trajectory error over eight KITTI sequences (544 s, 6.6 km), 2.3 times better than the GPS fixes that the filter receives
- **Cross-check.** Each error value calculated by this code and by **evo**; the run fails if the two differ by more than 10^{-6}
- **Tracking.** **0.886 MOTA** with 5 identity switches on the 11-sequence KITTI Tracking validation split with ground-truth boxes; 0.66 with raw PointRCNN detections and a score gate
- **Coupling test.** GPS removed for 4 s: ego error rises to 4.5 m, and track error follows with a correlation of 0.976
- **ESKF or UKF.** A UKF gives the same error at about 24 times the runtime, so the ESKF is used
- **Process noise.** A larger gyro process noise corrects the overconfident sequence 0033 (1.01 m to 0.53 m) but breaks the 1 degree attitude target on the tuning drive, so the released configuration is unchanged
- **Lower bound.** A centred moving average of 11 raw fixes has a lower error than the filter on all eight sequences (0.394 m against 0.532 m). It uses future data, so it is a lower bound for a causal filter
- **Limits.** GPS fixes synthesised from the OXTS reference; the clutter test fails

C++, ROS 2, Python, YOLO11s, ONNX Runtime | **mAP50 0.96 on 478 validation images**



Costmap (purple), planned path (green) and the rover in simulation



Retrained detector on a test image

- ROS 2 navigation stack in C++: LiDAR scans fill a 30 m by 30 m costmap (10 cm cells, 0.5 m inflation), and a map-memory node merges the local costmaps into a global grid
- **A*** on the global grid with replanning at 2 Hz; **Pure Pursuit** path following at 10 Hz
- Retrained **YOLO11s** on 3,916 labelled images from two merged data sets to find an orange hammer, a water bottle and a pickaxe
- mAP50 0.96 and mAP50-95 0.87 on 478 validation images (best epoch 83; early stopping ended training at 98 of 100); runs on the rover as a ROS 2 node from an **ONNX** export

REPARO: AI REPAIR ASSISTANT

[GitHub](#)

Python, FastAPI, Swift, Gemini, SerpAPI, Reactiv SDK | **Hack Canada 2026 Winner (\$5,000), Most Complex AI Hack Finalist**



A sourced replacement part opened inside the App Clip

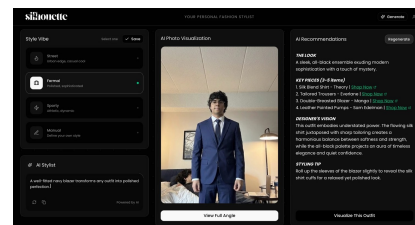
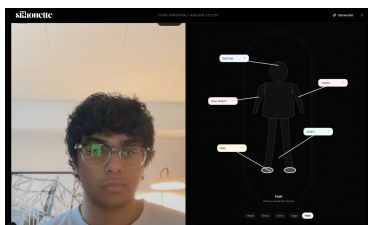
1st place in the Reactiv track, Hack Canada 2026

- One photo of a broken device to a structured repair plan in under 60 s, using component detection and **Gemini** reasoning on the image
- **FastAPI** service converts the LLM output into typed repair models, so the steps, cost estimates and tools always render in the same format
- Parts sourced through SerpAPI, Shopify storefront discovery and the Shopify product APIs into one checkout in a Swift Reactiv App Clip
- App Clip opens from a QR code or an NFC tag with no installation

SILHOUETTE: CURSOR FOR FASHION

[GitHub](#) | [Demo](#)

Node.js, Next.js, Express, MongoDB, Gemini, Overshoot SDK | **VC referral to continue as a proof of concept for Merian Brothers**



Live camera with body-part tags for the pieces to change

Generated outfit with shoppable recommendations

- User takes a photo with the live camera, selects the part of the outfit to change, and picks a style
- **Overshoot** analyses the photo and **Gemini** generates the new outfit on the image of the user, with links to real products
- Gemini pipelines on a MERN stack run inference, product recommendations and style constraints asynchronously, and return recommendations in under 2 s

Other work. [Floaty](#): MCP server with 68 tools for the Float Financial API. [HRR](#): LLM-guided tree search for document retrieval. [GitHub Repo Analyzer](#): PDF reports about a repository for different audiences.